



WELCOME





C++ STL : Pair & Vector

Agenda

C++ Bridge Course

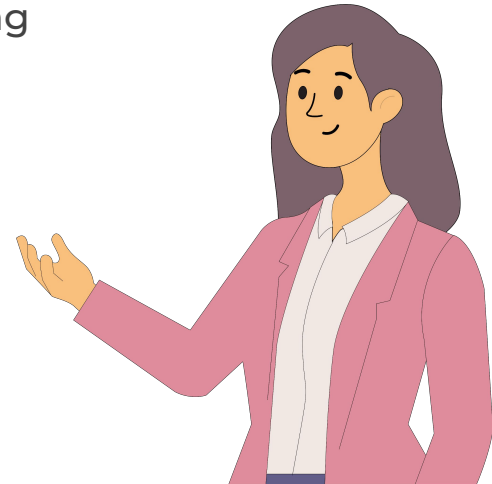
Introduction to
C++ STL

Pair

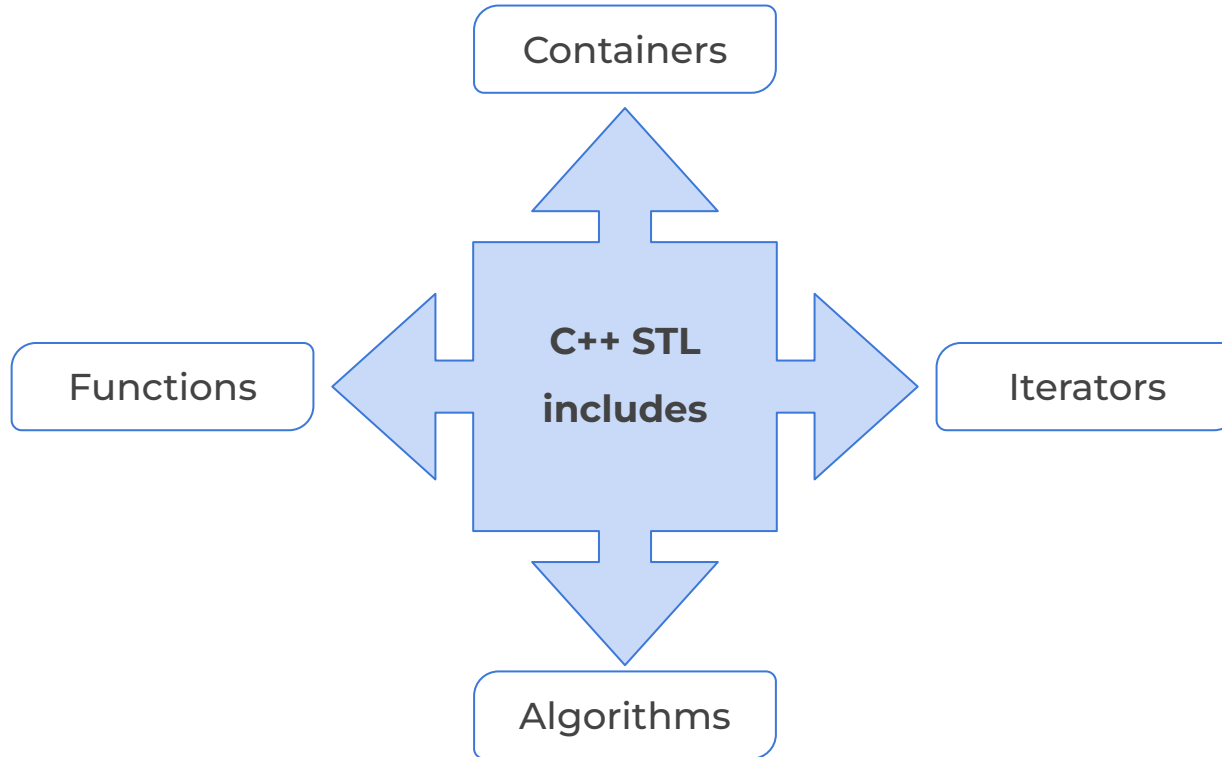
Vector

Introduction

- ✓ C++ STL (**Standard Template Library**) is a versatile collection of **template classes and functions** providing ready-to-use **data structures and algorithms for generic programming**.



Introduction





Pair Container

Introduction

What is a Pair in C++?

1

A pair in C++ is a container that holds two values together.

2

These values can be of the same or different types.

3

It is part of the C++ Standard Template Library (STL).

Declaring a Pair

A C++ pair stores two values together.

C++



```
pair<int, char> p1 = {1, 'A'};  
pair<int, string> p2 = make_pair(2, "Alice");
```

pair<type1, type2>

Accessing Elements in a Pair

C++



```
pair<int, char> p1 = {1, 'A'};  
cout << "First: " << p1.first << endl;  
cout << "Second: " << p1.second << endl;
```

Output



First: 1

Second: A

We use **.first** and **.second** to *access elements*.

Storing 3 or More Elements

Using Nested Pairs

C++ 

```
pair<int, pair<int, int>> p3 = {3, {4, 6}};  
cout << p3.first << endl;  
cout << p3.second.first << " " << p3.second.second << endl;
```

Output 

3
4 6

*In case of nested Pairs, multiple **.first** and **.second** can be used in above way.*

Array of Pairs

C++ 

```
pair<int, int> arr[] = {{4,7}, {5,2}, {6,9}, {8,1}};  
cout << arr[3].first << endl;
```

Output 

8

We can use array of pairs

Inserting and Modifying Elements in a Pair

Modifying .first and .second

C++



```
pair<int, string> student = {101, "John"};  
student.first = 201;  
student.second = "Michael";  
cout << student.first << " " << student.second;
```

Output

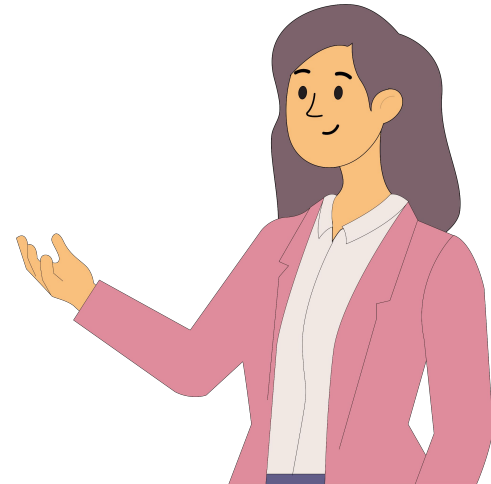


201 Michael

C++ pairs allow modification.

Problem Statement

- ✓ You are given **two pairs**, (a, b) and (x, y) .
- ✓ Compare the **first elements** (a and x):
 - If $a > x$, then (a, b) is the greater pair.
 - If $x > a$, then (x, y) is the greater pair.
- ✓ If $a = x$, compare the **second elements** (b and y) to determine the greater pair.



Quiz Time!





Vector

Introduction

Vector

In C++ STL, a **vector** is a **dynamic array** container offering **dynamic resizing, contiguous memory storage**, and **versatile functions** for managing **sequences of elements**.

Introduction

To better **comprehend the significance of vectors**,
let's consider the concept of arrays.

```
int arr[] = {4, 5, 6}
```

Size of the array = 3

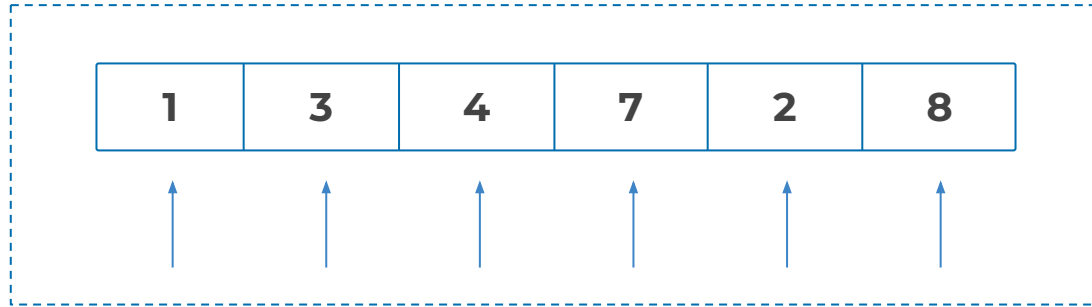
Introduction

Once an array is created, its size cannot be changed.

By using vectors, we can increase and decrease the size of the storage.



Introduction



In Vector, all the elements are **stored in contiguous memory locations.**



Declaration of a Vector

Declaration

Declaration with Initialization

Vector with predefined values.

C++

```
vector<int> v1 = {1, 3, 5, 7, 9};
```

v1 =

1

3

5

7

9

List with predefined values.

Python

```
v1 = [1, 3, 5, 7, 9]
```

v1 =

1

3

5

7

9

Declaration

Constructor initialization of vector

Creates a **vector of size 5**, all elements **set to 20**.

C++



```
vector<int> v2(5, 20);
```

v2 =

20

20

20

20

20

Declaration

Constructor initialization with one parameter

Creates a **vector of size 5** with default **values (0)**.

C++



```
vector<int> v3(5);
```

v3 =

0

0

0

0

0

Copying an Existing Vector

Creates a copy of v1.

C++ 

```
vector<int> v1 = {1, 3, 5, 7, 9};  
vector<int> v4(v1);
```

v4 =

1

3

5

7

9



Accessing Elements in a Vector



Accessing Elements Using Indexing

Both C++ vectors and Python lists allow element access using indexing.

C++

```
vector<int> v = {1, 3, 5, 7, 9};
```

	0	1	2	3	4
v =	1	3	5	7	9

Python

```
v = [1, 3, 5, 7, 9]
```

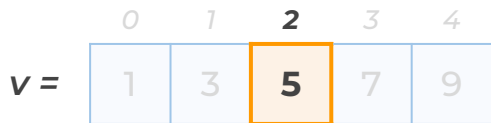
	0	1	2	3	4
v =	1	3	5	7	9

Accessing Elements Using Indexing

Both C++ vectors and Python lists allow element access using indexing.

C++

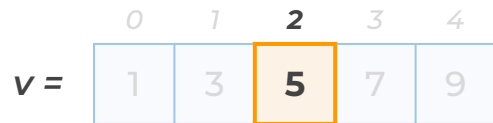
```
vector<int> v = {1, 3, 5, 7, 9};  
cout << v[2] << endl;
```



Accessing element at **index 2 (5)**

Python

```
v = [1, 3, 5, 7, 9]  
print(v[2])
```



Accessing element at **index 2 (5)**

Accessing Elements Using Indexing

Both C++ vectors and Python lists allow element access using indexing.

C++ 

```
vector<int> v = {1, 3, 5, 7, 9};  
cout << v[2] << endl;
```

Output 

5

Python 

```
v = [1, 3, 5, 7, 9]  
print(v[2])
```

Output 

5

Accessing First and Last Element

C++ vectors allow easy access to the first and last elements.

C++

```
vector<int> v = {1, 3, 5, 7, 9};  
cout << v.front() << endl;  
cout << v.back() << endl;
```

	0	1	2	3	4
v =	1	3	5	7	9

Accessing First and Last Element

C++ vectors allow easy access to the first and last elements.

C++

```
vector<int> v = {1, 3, 5, 7, 9};  
cout << v.front() << endl;  
cout << v.back() << endl;
```

	0	1	2	3	4
v =	1	3	5	7	9

Accessing First and Last Element

C++ vectors allow easy access to the first and last elements.

C++



```
vector<int> v = {1, 3, 5, 7, 9};  
cout << v.front() << endl;  
cout << v.back() << endl;
```

Output



1

9

Accessing Elements Using Loop

We can loop through the C++ vector utilizing indexing.

C++

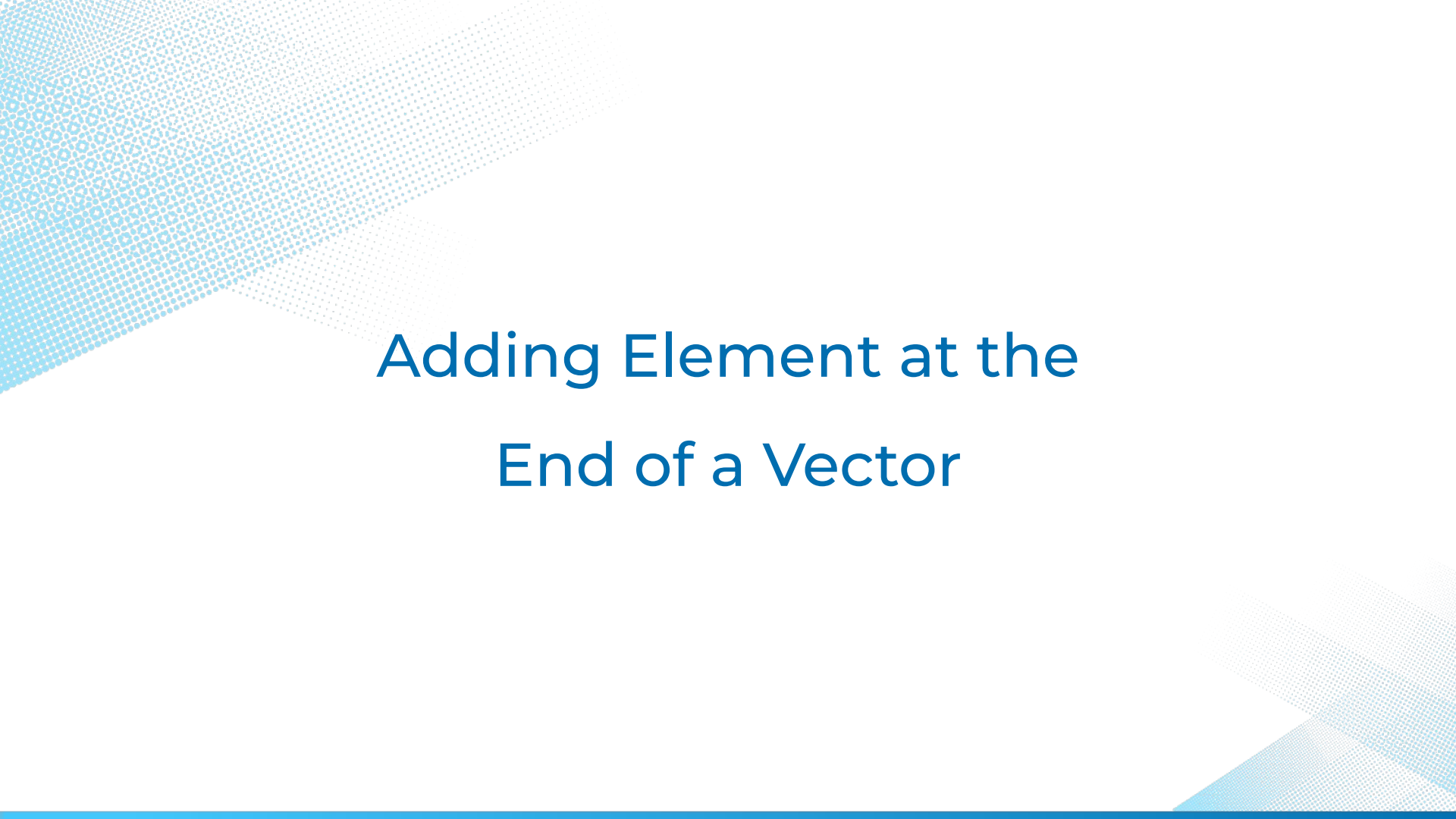


```
vector<int> v = {1, 3, 5, 7, 9};  
for (int i = 0; i < v.size(); i++) {  
    cout << v[i] << " ";  
}
```

Output



1 3 5 7 9



Adding Element at the End of a Vector

Adding Element at the End

C++



```
vector<int> v = {1, 3, 5, 7, 9};  
v.push_back(11);
```

v =

0	1	2	3	4
1	3	5	7	9

Python



```
v = [1, 3, 5, 7, 9]  
v.append(11)
```

v =

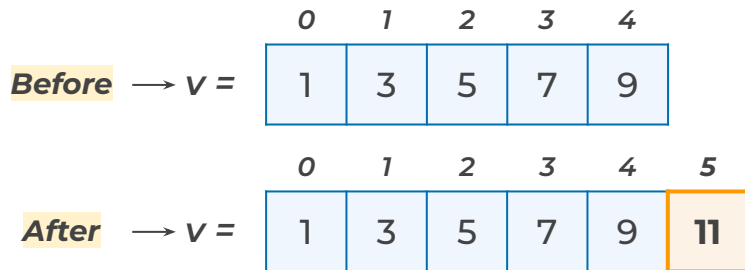
0	1	2	3	4
1	3	5	7	9

Adding Element at the End

C++



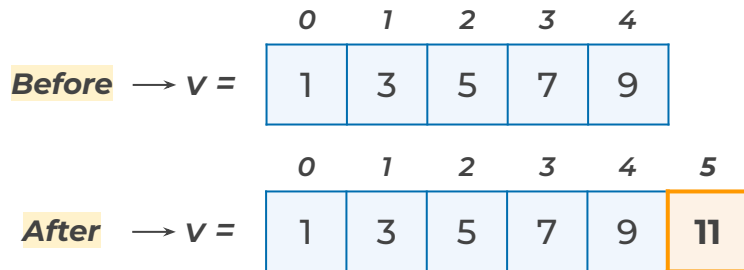
```
vector<int> v = {1, 3, 5, 7, 9};  
v.push_back(11);
```



Python



```
v = [1, 3, 5, 7, 9]  
v.append(11)
```



Efficiently Adding Complex Elements

C++



```
vector<pair<int, string>> v;  
v.emplace_back(1, "Alice");
```

- **`emplace_back()`** constructs elements in-place, *avoiding unnecessary copies*.
- In C++, **`emplace_back()`** is *more efficient* than **`push_back()`** when dealing with *complex objects*.

Efficiently Adding Complex Elements

C++



```
vector<pair<int, string>> v;  
v.emplace_back(1, "Alice")  
v.push_back({2, "Bob"});
```

- With **push_back**, you typically use `{ }` to *pass* a *fully constructed object*, whereas with **emplace_back**, you can *directly* supply the *constructor arguments* without `{ }`.

Adding Multiple Elements at the End

C++



```
vector<int> v = {1, 3, 5};  
vector<int> new_elements = {7, 9, 11};  
  
for (int i = 0; i < new_elements.size(); i++) {  
    v.push_back(new_elements[i]);  
}
```

v =

0	1	2
1	3	5

Python



```
v = [1, 3, 5]  
new_elements = [7, 9, 11]  
v.extend(new_elements)
```

v =

0	1	2
1	3	5

Adding Multiple Elements at the End

C++

</>

```
vector<int> v = {1, 3, 5};  
vector<int> new_elements = {7, 9, 11};  
  
for (int i = 0; i < new_elements.size(); i++) {  
    v.push_back(new_elements[i]);  
}
```

Before → v =

0	1	2
1	3	5

After → v =

0	1	2	3	4	5
1	3	5	7	9	11

Python

</>

```
v = [1, 3, 5]  
new_elements = [7, 9, 11]  
v.extend(new_elements)
```

Before → v =

0	1	2
1	3	5

After → v =

0	1	2	3	4	5
1	3	5	7	9	11



Other Functions in C++ Vector

Removing Element From Last

To “*remove*” element from last.

C++



```
vector<int> v = {1, 3, 5, 7};
```

v =

0	1	2	3
1	3	5	7

Python



```
v = [1, 3, 5, 7]
```

v =

0	1	2	3
1	3	5	7

Removing Element From Last

Removes last element (7).

C++



```
vector<int> v = {1, 3, 5, 7};  
v.pop_back();
```

Before → v =

0	1	2	3
1	3	5	7

After → v =

0	1	2
1	3	5

Removes last element (7).

Python



```
v = [1, 3, 5, 7]  
v.pop()
```

Before → v =

0	1	2	3
1	3	5	7

After → v =

0	1	2
1	3	5

Getting the Number of Elements

C++



```
vector<int> v = {1, 3, 5, 7};
```

v =

0	1	2	3
1	3	5	7

Python



```
v = [1, 3, 5, 7]
```

v =

0	1	2	3
1	3	5	7

Getting the Number of Elements

C++



```
vector<int> v = {1, 3, 5, 7};  
cout << v.size();
```

Output



4

Python



```
v = [1, 3, 5, 7]  
print(len(v))
```

Output



4

Removing Element from a Specific Position

To “**Remove**” a specific element based on its index.

C++



```
vector<int> v = {1, 3, 5, 7, 9};
```

	0	1	2	3	4
v =	1	3	5	7	9

Python



```
v = [1, 3, 5, 7, 9]
```

	0	1	2	3	4
v =	1	3	5	7	9

Removing Element from a Specific Position

*Removes element at **index 2** (5).*

C++



```
vector<int> v = {1, 3, 5, 7, 9};  
v.erase(v.begin() + 2);
```

Before → v =

0	1	2	3	4
1	3	5	7	9

After → v =

0	1	2	3
1	3	7	9

*Removes element at **index 2** (5).*

Python



```
v = [1, 3, 5, 7, 9]  
v.pop(2)
```

Before → v =

0	1	2	3	4
1	3	5	7	9

After → v =

0	1	2	3
1	3	7	9

Removing a Range of Elements

To remove multiple elements within a specified range.

C++

</>

```
vector<int> v = {1, 3, 5, 7, 9};  
v.erase(v.begin() + 1, v.begin() + 4);
```

v =

0	1	2	3	4
1	3	5	7	9

Python

</>

```
v = [1, 3, 5, 7, 9]  
del v[1:4]
```

v =

0	1	2	3	4
1	3	5	7	9

Removing a Range of Elements

*Removes elements from **index 1** to **3**.*

C++



```
vector<int> v = {1, 3, 5, 7, 9};  
v.erase(v.begin() + 1, v.begin() + 4);
```

Before → v =

0	1	2	3	4
1	3	5	7	9

After → v =

0	1
1	9

*Removes elements from **index 1** to **3**.*

Python



```
v = [1, 3, 5, 7, 9]  
del v[1:4]
```

Before → v =

0	1	2	3	4
1	3	5	7	9

After → v =

0	1	2	3
1	9		

Inserting Elements at a Specific Position

To insert 4 at index 2.

C++



```
vector<int> v = {1, 3, 5, 7, 9};  
v.insert(v.begin() + 2, 4);
```

v =

0	1	2	3	4
1	3	5	7	9

To insert 4 at index 2.

Python



```
v = [1, 3, 5, 7, 9]  
v.insert(2, 4)
```

v =

0	1	2	3	4
1	3	5	7	9

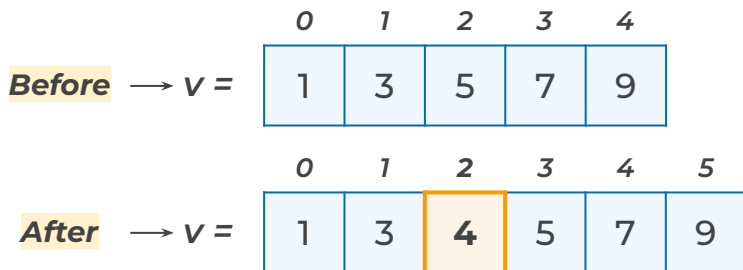
Inserting Elements at a Specific Position

Inserts element 4 to index 2.

C++

</>

```
vector<int> v = {1, 3, 5, 7, 9};  
v.insert(v.begin() + 2, 4);
```

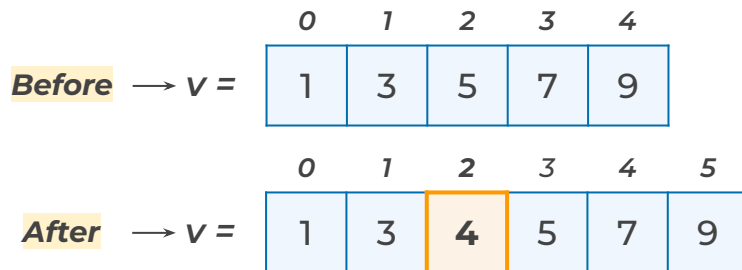


Inserts element 4 to index 2

Python

</>

```
v = [1, 3, 5, 7, 9]  
v.insert(2, 4)
```



Inserting Multiple Elements

Inserts [10, 11] at index 2.

C++



```
vector<int> v = {1, 3, 5, 7, 9};
```

v =

0	1	2	3	4
1	3	5	7	9

Inserts [10, 11] at index 2.

Python



```
v = [1, 3, 5, 7, 9]
```

v =

0	1	2	3	4
1	3	5	7	9

Inserting Multiple Elements

Inserts [10, 11] at index 2.

C++

</>

```
vector<int> v = {1, 3, 5, 7, 9};  
vector<int> v2 = {10, 11};  
v.insert(v.begin() + 2, v2.begin(), v2.end());
```

Before → v =

0	1	2	3	4
1	3	5	7	9

After → v =

0	1	2	3	4	5	6
1	3	10	11	5	7	9

Inserts [10, 11] at index 2.

Python

</>

```
v = [1, 3, 5, 7, 9]  
v[2:2] = [10, 11]
```

Before → v =

0	1	2	3	4
1	3	5	7	9

After → v =

0	1	2	3	4	5	6
1	3	10	11	5	7	9

Reversing vector in C++

To reverse elements

C++



```
vector<int> v = {1, 3, 5, 7, 9};
```

	0	1	2	3	4
v =	1	3	5	7	9

Python



```
v = [1, 3, 5, 7, 9]
```

	0	1	2	3	4
v =	1	3	5	7	9

Reversing vector in C++

Reverses all elements of vector

C++

</>

```
vector<int> v = {1, 3, 5, 7, 9};  
reverse(v.begin(), v.end());
```

Before → v =

0	1	2	3	4
1	3	5	7	9

After → v =

0	1	2	3	4
9	7	5	3	1

Reverses all elements of list

Python

</>

```
v = [1, 3, 5, 7, 9]  
v.reverse()
```

Before → v =

0	1	2	3	4
1	3	5	7	9

After → v =

0	1	2	3	4
9	7	5	3	1

Removing All Elements

Removing all elements of vector

C++ 

```
vector<int> v = {1, 3, 5, 7};
```

v =

0	1	2	3
1	3	5	7

Removing all elements of list

Python 

```
v = [1, 3, 5, 7]
```

v =

0	1	2	3
1	3	5	7

Removing All Elements

Removes all elements of vector

C++

</>

```
vector<int> v = {1, 3, 5, 7};  
v.clear();
```

Before → v =

0	1	2	3
1	3	5	7

After → v =

0	1	2	3

Removes all elements of list

Python

</>

```
v = [1, 3, 5, 7]  
v.clear()
```

Before → v =

0	1	2	3
1	3	5	7

After → v =

0	1	2	3

Swapping Two Vectors

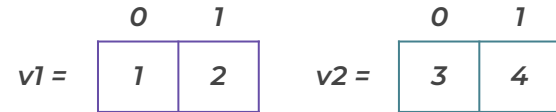
Swap to have all values of v1 in v2 and vice versa

C++

```
vector<int> v1 = {1, 2};  
vector<int> v2 = {3, 4};
```

**Python**

```
v1 = [1, 2]  
v2 = [3, 4]
```



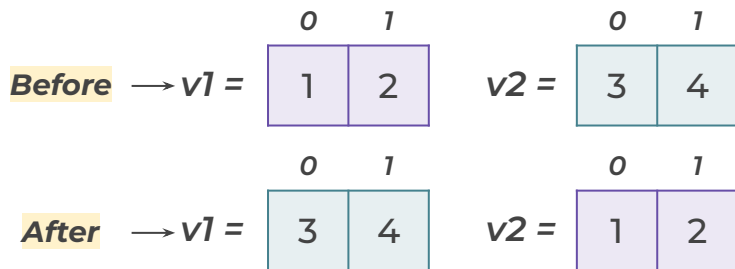
Swapping Two Vectors

v1 becomes {3, 4}, v2 becomes {1, 2}.

C++

</>

```
vector<int> v1 = {1, 2};  
vector<int> v2 = {3, 4};  
v1.swap(v2);
```

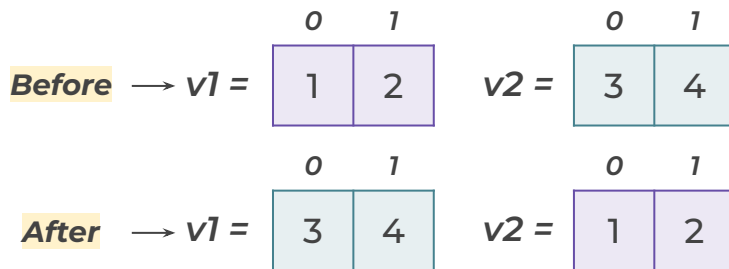


v1 becomes {3, 4}, v2 becomes {1, 2}.

Python

</>

```
v1 = [1, 2]  
v2 = [3, 4]  
v1, v2 = v2, v1
```



Operation Costs

Adding at the End (`push_back/append`):

- On average, 1 step to add the element.

Removing from the End (`pop_back`):

- Steps: 1 step to remove the element.

Operation Costs

Inserting in the Middle (insert):

- **Scenario:** An element is inserted somewhere other than the end.
- **Steps:** Approximately N steps are needed (where N is the number of elements), since every element after the insertion point must be moved one position over.

Deleting from the Middle (erase):

- **Steps:** Approximately N steps (where N is the number of elements), as each element following the removed element must be shifted to close the gap.



Iterators in C++ vs Python



Iterators in C++ vs Python



What are Iterators?

- **C++ iterators** allow traversing through **STL** containers like **vector**, **list**, etc.
- **Python iterators** work with sequences like **list**, **tuple**, **set**, and support direct iteration using loops.
- Iterators help access elements **without using indexes**.

Accessing Start and End of a Vector

C++



```
vector<int> v = {1, 3, 5, 7, 9};
```

```
vector<int>::iterator it = v.begin();
```

```
cout << "First element: " << *it << endl;
```

```
vector<int>::iterator it_end = v.end();
```

```
it_end--;
```

```
cout << "Last element: " << *it_end << endl;
```

`v.begin()` → Points to the first element.

Accessing Start and End of a Vector

C++



```
vector<int> v = {1, 3, 5, 7, 9};  
  
vector<int>::iterator it = v.begin();  
cout << "First element: " << *it << endl;  
  
vector<int>::iterator it_end = v.end();  
it_end--;  
cout << "Last element: " << *it_end << endl;
```

it* → used to **dereference an iterator to
access the value it points to.

Accessing Start and End of a Vector

C++



```
vector<int> v = {1, 3, 5, 7, 9};  
  
vector<int>::iterator it = v.begin();  
cout << "First element: " << *it << endl;  
  
vector<int>::iterator it_end = v.end();  
it_end--;  
cout << "Last element: " << *it_end << endl;
```

`v.end()` → Points past the last element.

Accessing Start and End of a Vector

C++



```
vector<int> v = {1, 3, 5, 7, 9};  
  
vector<int>::iterator it = v.begin();  
cout << "First element: " << *it << endl;  
  
vector<int>::iterator it_end = v.end();  
it_end--;  
cout << "Last element: " << *it_end << endl;
```

`v.end()-1` gets the last element.

Reverse Iteration

C++



```
vector<int> v = {1, 3, 5, 7, 9};  
  
vector<int>::reverse_iterator rit = v.rbegin();  
cout << "Last element (using rbegin): " << *rit << endl;  
  
vector<int>::reverse_iterator rit_end = v.rend();  
rit_end--;  
cout << "First element (using rend-1): " << *rit_end <<  
endl;
```

`v.rbegin()` → Points to the last element.

Reverse Iteration

C++



```
vector<int> v = {1, 3, 5, 7, 9};

vector<int>::reverse_iterator rit = v.rbegin();
cout << "Last element (using rbegin): " << *rit << endl;

vector<int>::reverse_iterator rit_end = v.rend();
rit_end--;
cout << "First element (using rend-1): " << *rit_end <<
endl;
```

***v.rend()** → Points before the first element.*

Reverse Iteration

C++

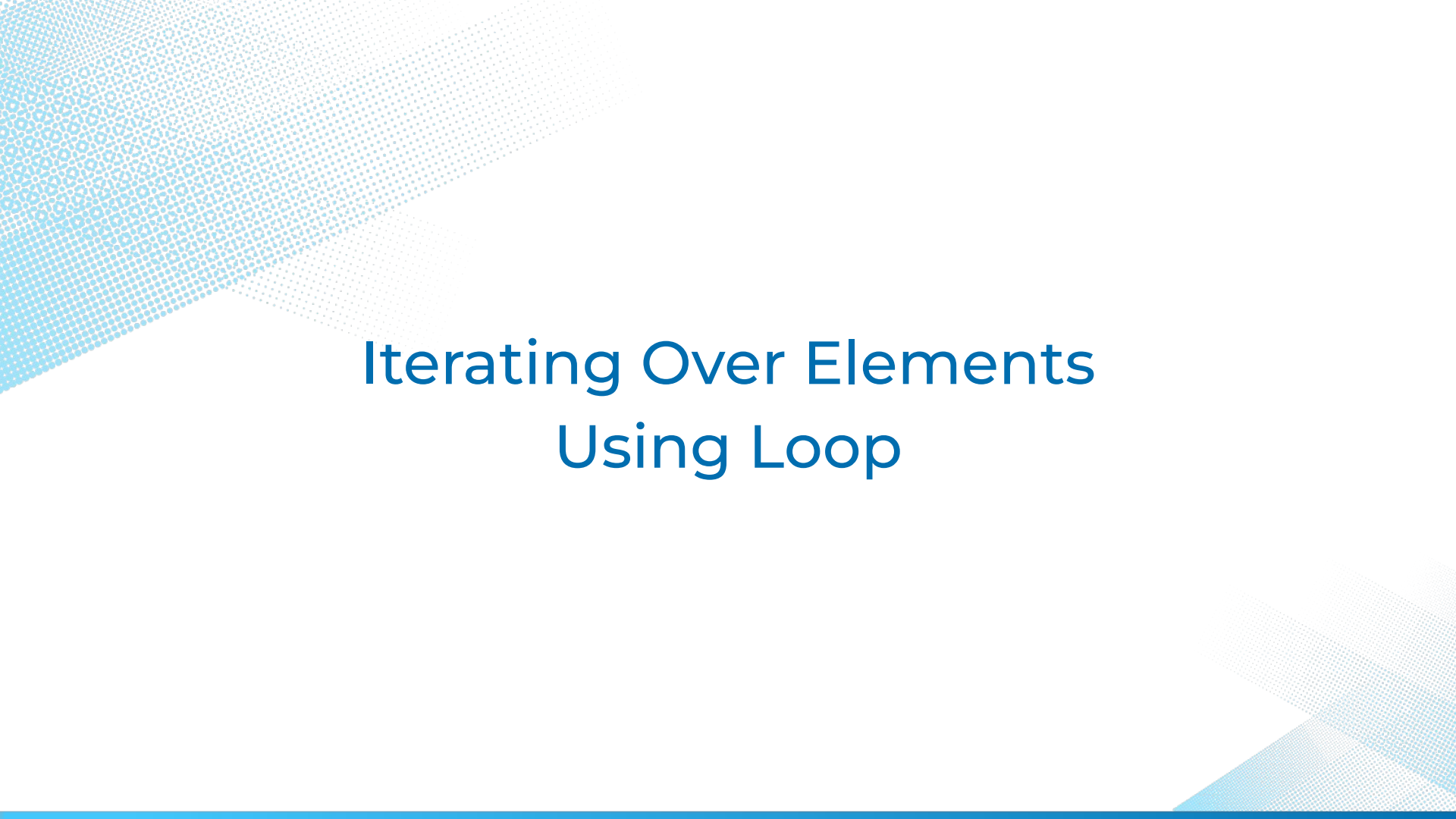


```
vector<int> v = {1, 3, 5, 7, 9};

vector<int>::reverse_iterator rit = v.rbegin();
cout << "Last element (using rbegin): " << *rit << endl;

vector<int>::reverse_iterator rit_end = v.rend();
rit_end--;
cout << "First element (using rend-1): " << *rit_end <<
endl;
```

so `(v.rend()-1)` gets the first element.



Iterating Over Elements Using Loop

Iterating Over Elements Using Loop

C++



```
vector<int> v = {1, 3, 5, 7, 9};  
for (vector<int>::iterator it = v.begin(); it  
    != v.end(); it++) {  
    cout << *it << " ";  
}
```

Output



1 3 5 7 9

Iterating Using begin() and end().

Iterating Over Elements Using Loop

C++



```
vector<int> v = {1, 3, 5, 7, 9};  
for (vector<int>::reverse_iterator rit =  
v.rbegin(); rit != v.rend(); rit++) {  
    cout << *rit << " ";  
}
```

Output



9 7 5 3 1

Iterating Using rbegin() and rend() (Reverse Order).

Using Auto For Simplified Iterator Declaration

C++



```
vector<int> v = {1, 3, 5, 7, 9};  
  
for (auto it = v.begin(); it != v.end(); it++) {  
    cout << *it << " ";  
}
```

Using Range-Based Loops for Simplicity

C++



```
vector<int> v = {1, 3, 5, 7, 9};  
for (int num : v) {  
    cout << num << " ";  
}
```

Output



1 3 5 7 9

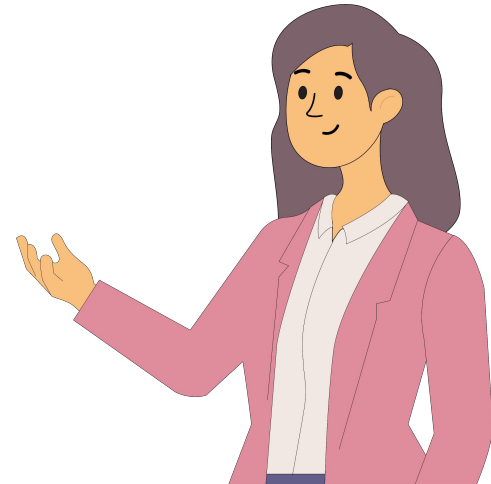
Quiz Time!



Alternate Numbers from 1 to n

Problem Statement

- ✓ Write a function that takes **integer N** as **input** and **returns a vector** containing every alternate number from **1** to **N**, starting with **1**.



Key Takeaways part(1/2)



C++ STL:

A powerful library with **containers**, **functions**, **iterators**, and **algorithms** for efficient coding.



Pair:

Stores two values of the same or different types, accessed using **.first** and **.second**.
Supports nesting and arrays of pairs.



Key Takeaways part(2/2)



Vector:

A dynamic array that allows resizing, **element access** (`[]`), and **modifications** like **push_back**, **erase**, and **insert**.



Operations:

Supports **iterators** (`begin()`, `end()`), **safe access**, **swapping**, **reversing**, and memory-efficient insertion with **emplace_back()**.





THANK YOU





ALL THE BEST



Safe Access Using .at() in C++

- ✓ In C++, `v.at(index)` is safer than `v[index]` as it throws an exception if the index is out of range, whereas `v[index]` can **cause undefined behavior**.
- ✓ Python does not have `.at()`, but list indexing already raises an error when out of bounds.

Safe Access Using .at() in C++

C++



```
vector<int> v = {1, 3, 5, 7, 9};  
cout << v.at(2) << endl;
```

Output



5